TLE / ICT 9
FOURTH QUARTER

LESSON 4

What are Events?

Events are actions that are triggered either by mouse click, function calls, keyboard strokes, loading of images, submitting an HTML form, etc.

Events are usually used in connection with functions. Functions will not work without a force that will trigger it.

Table of Event Triggers

Attribute	To trigger the event
onabort	The loading of an image is interrupted.
onblur	An element loses focus.
onchange	The user changes the content of a field.
onclick	Mouse click of an object.
ondblclick	Double mouse click on an object.
onerror	An error occurs when loading a document or image.
onfocus	An element gets focus.
onkeydown	A keyboard key is pressed.
onkeypress	A keyboard key is held down.
onkeyup	A keyboard key is released.

Table of Event Triggers

Attribute	To trigger the event
onload	A page or image is finished loading.
onmousedown	A mouse button is pressed.
onmousemove	A mouse is moved.
onmouseout	The mouse is moved off an element.
onmouseover	The mouse is moved over an element.
onmouseup	Ana mouse button is pressed.
onreset	The reset button is clicked.
onresize	A window or frame is resized.
onselect	Text is selected.
onsubmit	The submit button is clicked.
onunload	The user exits the page.

- **onload and onunload**

The onload and onunload events are triggered when a user enters or exits a Web page. The onload event can be used to check the Web browser version type and version. The onload and onunload events can also be used to store and retrieve cookies. A cookie is a message given to a Web browser

- **onfocus, onblur and onchange**

  These attributes are often used with the validation of fields in a form.

- **onsubmit**

  This attribute is used to validate all fields in the form before submitting it.

Example using onsubmit:

```
<form method="post" action="trial.htm"
onsubmit="return checkFields()">
```

This is how the onsubmit attribute is usually written. When the submit button is clicked, it will call the function checkFields(). The function will check all the fields in the form. If everything is cleared, it will return a 'true' so the form will be submitted; otherwise, it will return a 'false' so the form will be cancelled.

- **onmouseover and onmouseout**

These attributes are usually used for animation purposes. Take for example, a button or link in a Web page. When the mouse is over it, it will display a different image, however, when you remove the mouse, the button is set back to what it was.

- **onclick**

This is usually used to connect you to another webpage or to trigger an event call. Take for example, a button in a Web page. If that button is pressed, it will trigger a function that will call a pop-up button.

What are Popup Boxes?

Popup boxes are those windows that pop-out of your screen. They could be before a page is fully loaded, effect of an event on a page or before leaving a page. JavaScript has three kinds of popup boxes; alert box, user-input prompt box and confirm box. These popup boxes can be useful in warning the user, gathering user input or simply to verify something.

- **Alert Box**

This is usually used when you want to make sure that the user reads your message such as warning or a greeting.

Syntax:

```
alert("message");
```

Note:

```
<!DOCTYPE html>
<html>
<head><title>Alert Box</title>
  <script type="text/JavaScript">
      function show_alert()
       {alert("WARNING!  WARNING!");}
  </script>
</head>
<body>
   <input type="button" onclick="show_alert()"
     value="show alert box">
</body>
</html>
```

Note: When the button is clicked, an alert message in the form of a popup box will be shown.

- **User-Input Prompt Box**

This is usually used when you want data to be prompted first before the user can enter a page. The user will have to choose either "ok" or "cancel" to continue. If the user chooses "ok", the prompt box will return the encoded value, else, it will return null.

Syntax:

```
prompt("message","defaultvalue");
```

```
<!DOCTYPE html>
<html>
<head><title>Prompt Box</title>
  <script type="text/JavaScript">
      function show_prompt()
       {var name=prompt("Please enter your name:","nameless");
          if(name!=null && name!="")
       {document.write("Welcome" + name + "!
          Please come in");}}
  </script>
</head>
<body>
   <input type="button" onclick="show_prompt()" value="show prompt">
</body>
</html>
```

Note: When the button is clicked, a prompt box will be shown asking for the user input. The user input will be saved on the variable name. The user input will then be validated.

- **Confirm Box**

This is usually used when you want the user to verify or accept something. The user will have to choose either "ok" and "cancel". If "ok" was chosen, the confirm box will return true; else, it will return false.

Syntax:

```
confirm("message");
```

```
<!DOCTYPE html>
<html>
<head><title>Confirm Box</title>
  <script type="text/JavaScript">
      function show_confirm()
       {var ans=confirm("Do you want to proceed?");
          if(ans==true)
       {document.write("You have entered stage 2");}
          else
       {document.write("You are still at stage 1");
</script></head><body>
   <input type="button" onclick="show_confirm()" value="show confirm box">
</body>
</html>
```

Note: When the button is clicked, a confirmation box will be shown asking for confirmation if it will proceed. The choice will be saved on the variable ***ans.*** The choice will then be used to activate the proper block of code.

What is a Variable?

A variable is a temporary storage of values and expressions. Expressions are formulae that use mathematical symbols and numbers or a way to assign values to a variable. For example, x=1+3 is an expression and the value o=f x will hold the sum of one plus three which is four.

In declaring a variable, there are certain rules to follow:

- Variable names are case-sensitive – an uppercased variable such as “X” is different from a lowercase variable “x”.
- Variable names must begin with a letter or underscore (“_”).
- The rest of the name must consist of letters, numbers or underscores.
- Avoid naming variables using reserved words. Reserved words are words reserved by JavaScript for it own use.

Take note: if you do not declare a variable ang you give the variable a value, the variable will automatically be declared.

- Variables can hold numbers, expressions or string literals.
- String literals are text enclosed in pairs of quotation marks. Numeric digits are inside the quotation marks will be considered as as string and will lose then its numeric value

- If the characters are not enclosed in quotation marks, JS might interpret the characters as an object or a function.
- To declare a variable, use the keyword ***var*** then the variable name equal to its value/expression.
- JS variables are loosely typed, so data types are not required to be defined in advance. This feature is an advantage since it can adapt its data type from the kind of value assigned to it. It can be an integer, string Boolean, etc. It can change during runtime; however, be careful in assigning values because it you make a mistake, it will be hard to detect such kind of errors.

- Boolean is a fundamental data type in programming that represents one of two possible values: **true** or **false**. It is used to represent logical truth values, acting as the foundation for decision-making in code, such as determining which path a program should follow.

Example of variable declaration:

```
var x = 5;
var y = x+5;
var fan = “Standard”;
var isDone = false;
```

- JS uses the same arithmetic operations as in most programming languages, like C. An arithmetic operator is a symbol used to tell the expression what kind of operation will be done.

Operator	Description	Example
+	Addition	Sum=x+y
-	Subtraction	Diff=x-y
*	Multiplication	Times=x*y
/	Division	Div=x/y
%	Modulus	Mod=x%y
++	Increment by 1	Inc=x++
--	Decrement by 1	Dec=x--

Modulus is performed by the **remainder operator (%)**, which calculates the remainder after one number (the dividend) is divided by another (the divisor)

- The addition symbol ('+') can also be used to add strings or text values together – this is called concatenation. However, if you add a string and a numerical value, the result will still be a string.

```
SentenceA = "Good"
SentenceB = "Morning"
SentenceC = "SentenceA + "" SentenceB + 2 + 4

Output:
Good Morning 24
```

- Remember the order of precedence when dealing with expressions. Innermost parentheses come first, then the outer parentheses are evaluated followed by multiplication, division or modulus operation. The addition or subtraction symbol is taken into consideration last.

X=5+3/2*6
=**14**
Y=(5+3)/2*6
=**24**

- The ++ and – operator can be used in several different ways; however, the answer will depend on how you use it. If you use it independently, x++; it will have the same result as ++x. If these operators are used in a formula or an expression, they will have an impact on the result. Avoid using these operators if you do not fully understand how to use them

Example using ++ and -- operators

Let x=5, y=7 and z=0

1) x = y++
   result: x=7, y=8

2) x = ++y
   result: x=8, y=8

3) z =x++ * ++y
   result: x=6, y=8, z=40

1st example: x is 1st assigned the value of y which is 7. Then y is incremented.

2nd example: x is assigned the value of y that had been incremented.

Example using ++ and -- operators

Let x=5, y=7 and z=0

1) x = y++
   result: x=7, y=8

2) x = ++y
   result: x=8, y=8

3) z =x++ * ++y
   result: x=6, y=8, z=40

3rd example: x is 1st evaluated. x will still have the value 5 and y will have an incremented value of 8. When x is multiplied to y, it will yield 40 which will be stored in variable z. That's only when the value of x will be incremented to 6.

What are Comparison Statements?

Comparison statements are used to evaluate the quality or the difference between variables or values.

Operand	Example	Results
==	x==y	Returns true if x is equal to y
===	x===y	Returns true if x is equal to y and the data are of the same type
!=	x!=y	Returns true if x is not equal to y
!==	x!==y	Returns true if x is not equal to y and/or the data are not of the same type.

Comparison Statements

Operand	Example	Results
>	x>y	Returns true if x is greater than y
<	x<y	Returns true if x is less than y
>=	x>=y	Returns true if x is greater than or equal to y
<=	x<=y	Returns true if x is less than or equal to y

Logical Operators

Logical operators are used to determine the logic between expressions, variables, or values. It is usually used by two or more expressions to satisfy the needed conditions.

Operator	Description	Example	Results
&&	And	(x==y) && (x>3)	Returns true if both expressions are true
||	Or	(x==y) || (x>3)	Returns true if at least one of the expression is true
!	Not	! (x==y)	Returns true if x is not equal to y

JS also has its own conditional operator that automatically assigns a value depending on the result of the condition.
Syntax for conditional statements:

```
variablename=(condition)?value1:value2
```

Example using conditional statement:

```
myFavorite=(color=="pink")?"Terrific":"Change color";
```

In the example, color=="pink" is the conditional statement. If the variable color has the same value as pink, it will return the string "Terrific" and save its value to myFavorite. Otherwise, myFavorite will have the value of "Change color".

Conditional statements are used to perform different operations based on the set conditions. There are different kinds of conditional statements. These are:

- **if** statement – used to check if a specified condition is true.
- **if-else** statement – used to check if the condition is true and performs an action if the condition is false.
- **if-else if.else** statement – used to check if one or more blocks of code can be executed.
- **switch** statement – used if you want to select one of many blocks of code to be executed.

Example using the if block:

```
<script type="text/JavaScript">
//Buy the orange only if you have exactly 5 coins

    if (coins==5)
   {document.write("<b>Buy the Orange!</b>");}
</script>
```

Note: the term block is used instead of statement because a block is composed of several statements

The **if** block will execute the codes that are within the set of braces, if the condition is satisfied. Additionally, if the code to be executed is only made up of a single line, there is no need to enclose it in braces.

Example using the if-else if-else block:

```
<script type="text/JavaScript">
//Buy the orange if you have exactly 5 coins, if you have less than 4
//coins, buy the apple instead.  Otherwise, don't buy anything.

if (coins==5)
{document.write("<b>Buy the Orange</b>");}
else if (coins<4)
{document.write("<b>Buy the Apple</b>");}
else
{document.write("<b>Buy nothing</b>");
</script>
```

Note:
The else if statement executes codes other than the prior conditions. The else statement will be the last choice – that is, if there are no conditions that were met, it will fall under the else block.

The JS Switch statement function like the if statement. It is a conditional statement; however, only one variable is taken into consideration and bears the condition. You can also use it to select one or more blocks of code to be executed. To prevent it from continuously executing other blocks of code after it, you must use the break statement.